

Token Auth

Key Concepts

- **Token Auth** — server generates a random opaque string on login; client sends it on every request
- **Opaque token** — random string with no built-in meaning (unlike JWT which has claims)
- **Stateful** — server must store token→user mapping in DB to validate
- **Revocation** — easy; just delete the token from the DB

How It Works

1. User logs in with username + password
2. Server generates a random token (e.g. `a3f8c2d9e1b74f6a...`), stores it in DB with user mapping
3. Returns token to client
4. Client stores token, sends it on every request: `Authorization: Bearer a3f8c2d9e1b74f6a...`
5. Server looks up token in DB → finds user → authenticates

```
Login:
Client --> POST /login (user+pass) --> Server
                                generate token
                                store in DB: token → userId
Client <-- { token: "a3f8c2..." } <-- Server

Subsequent requests:
Client --> GET /api/data (Authorization: Bearer a3f8c2...) --> Server
                                DB lookup: token
                                → userId
Client <-- 200 OK <-- Server
```

Token Auth vs JWT

	Token Auth	JWT
Token content	Random opaque string	Structured payload (claims)
Server DB needed	Yes — every request hits DB	No — stateless verification
Revocation	Easy — delete from DB	Hard — must wait for expiry
Scalability	Harder — shared DB required	Easy — no shared state
Transparency	Opaque — server only knows meaning	Self-describing — anyone can decode

Industry Examples

- **GitHub Personal Access Tokens** — opaque tokens stored server-side, scoped to specific permissions
- **Stripe API Keys** — random tokens, DB-backed, easy to revoke per client
- **Django REST Framework** — default token auth uses DB-backed opaque tokens

Interview Questions

Q: How does Token Auth differ from JWT?

Token Auth uses a random opaque string with no built-in meaning — the server must look it up in the DB on every request. JWT is self-contained with claims, so the server verifies locally without a DB lookup. Token Auth is stateful; JWT is stateless.

Q: What's the advantage of Token Auth over JWT?

Revocation is instant — just delete the token from the DB. With JWT you can't invalidate a token until it expires. For use cases like API keys where you need immediate revoke capability, opaque tokens are better.

Q: What's the downside of Token Auth at scale?

Every request hits the DB for token lookup. At high traffic this becomes a bottleneck. Mitigation: cache the token→user mapping in Redis to avoid repeated DB hits.